# Compatibilidad de navegadores y criterios de instalación

## PWA offline-first para reportes de servicios técnicos

**Proyecto integrador · UTM**
**Versión:** 1.0
**Fecha:** 28 de octubre de 2026
**Entregable:** Día 2 — Compatibilidad, limitaciones y checklist de instalación

---

## Resumen

La aplicación se ejecuta sobre dos motores: Chromium en Android y escritorio, y WebKit en iOS. Ambos soportan Service Worker, IndexedDB y geolocalización, que son las tres capacidades que el flujo necesita. Las diferencias aparecen en la instalación, en la sincronización en segundo plano y en la política de almacenamiento.

Tres limitaciones de iOS cambian el diseño: no existe `beforeinstallprompt`, no existe Background Sync y Safari puede purgar el almacenamiento de un sitio que no se instala ni se visita durante varios días. La app responde con una guía de instalación propia, un botón de sincronización manual y la recomendación de instalar y sincronizar pronto. En Chromium se usa el prompt nativo, pero la lógica de sincronización es la misma para no mantener dos comportamientos.

---

## 1. Alcance y método

La revisión cubre las capacidades que el proyecto usa o decide no usar:

- Service Worker: registro, ciclo de vida e intercepción de `fetch`.
- IndexedDB: persistencia de reportes y cuota.
- Instalación: manifest, prompt en Chromium y guía en iOS.
- Sincronización: evento `online`, botón manual y Background Sync.
- APIs del dispositivo: geolocalización y cámara.
- Actualización: cómo llega una versión nueva al usuario.

El objetivo de compatibilidad es **iOS 16.4 o superior** y una versión reciente de Chrome en Android. iOS 16.4 es la primera versión que habilita notificaciones push para PWA instaladas, aunque el proyecto no las use. La app no depende de esa función.

El estado de cada prueba se reporta con tres valores:

- **Implementado:** el código existe y cubre el caso.
- **Verificado en escritorio:** se probó en Chromium y en las herramientas del navegador.
- **Verificado en Android:** se probó en un teléfono real por HTTPS con Ngrok.

- **No verificado en iOS:** el código y la guía existen, pero no hay iPhone disponible para una prueba física.

---

## 2. Matriz comparativa

| Característica | Chromium / Chrome Android | WebKit / Safari iOS | Decisión en el proyecto |
|---|---|---|---|
| Instalación | `beforeinstallprompt` disponible; prompt nativo del navegador | No existe el evento; se usa **Compartir → Agregar a inicio** | Botón propio en Android y diálogo con los tres pasos en iOS. |
| Detección de app instalada | `matchMedia('(display-mode: standalone)')` | `navigator.standalone` además del modo de visualización | Detección combinada para no mostrar la ayuda dos veces. |
| Persistencia | IndexedDB con cuota amplia; `navigator.storage.persist()` | ITP puede purgar el almacenamiento de sitios sin interacción; las PWA instaladas están exentas | Instalar, sincronizar pronto y ofrecer sincronización manual visible. |
| Background Sync | `SyncManager` disponible | **No soportado** | Evento `online`, sincronización al arrancar y botón manual; no se usa Background Sync para mantener paridad. |
| Notificaciones push | Soporte completo | Solo con la PWA instalada desde iOS 16.4 | Fuera de alcance; no bloquea la captura ni la sincronización. |
| Service Worker | Soporte completo, incluidos módulos | Soporte completo; los módulos ESM en el SW no son fiables | `sw.js` clásico generado por `injectManifest`, sin imports ESM en runtime. |
| Estrategia de caché | Cache API y Workbox sin diferencias | Cache API con límites de cuota más opacos | Precaching del shell, SWR para estáticos y `NetworkOnly` para la API. |
| Geocodificación | GPS con HTTPS y permiso | GPS con HTTPS, permiso y gesto del usuario | Botón explícito, estado en vivo y campo de referencia manual como respaldo. |
| Cuota de almacenamiento | Proporcional al espacio libre del disco | Más restrictiva y variable; no conviene asumir un mínimo | `navigator.storage.estimate()` solo para diagnóstico; el diseño no depende de la cuota. |
| Actualización | El SW nuevo toma control con `skipWaiting` | Mismo ciclo, con más cautela en la caché | `autoUpdate`, limpieza de cachés viejas y aviso de recarga. |
| Pantalla completa | `display: standalone` | `display: standalone` más metas de Apple | Manifest y metas `apple-mobile-web-app-*`; `safe-area-inset` en la barra inferior. |

---

## 3. Limitaciones críticas de iOS

### 3.1 Purga de almacenamiento por ITP

Intelligent Tracking Prevention aplica un límite de siete días al almacenamiento escribible por script en sitios que no se instalan y que no reciben interacción. Ese almacenamiento incluye IndexedDB, que es donde viven los reportes pendientes. Si el técnico captura sin red y no vuelve a abrir la app en una semana, el reporte puede desaparecer antes de sincronizarse.

La app no puede desactivar ITP. Puede reducir la exposición:

1. Instalar la PWA en la pantalla de inicio. Las apps instaladas quedan fuera de esa purga.
2. Sincronizar en cuanto haya red, sin esperar al final del día.
3. Pedir persistencia con `navigator.storage.persist()`. Es una solicitud; el navegador puede negarla.
4. Mostrar el estado pendiente y un botón de sincronización visible, para que el técnico sepa que hay datos sin enviar.

**Riesgo residual:** si el usuario nunca instala la app, nunca la abre y nunca sincroniza, el dato local puede perderse. No existe una API web que garantice lo contrario. La mitigación es de comportamiento, no una certeza técnica.

### 3.2 Ausencia de `beforeinstallprompt`

Safari no dispara el evento que permite mostrar un prompt de instalación desde la app. El usuario debe usar el menú Compartir y elegir **Agregar a inicio**. La app detecta iOS y muestra un diálogo con los tres pasos, sin intentar simular un prompt que el sistema no ofrece.

En Chromium, el evento se captura, se guarda en `deferredPrompt` y se muestra un botón **Instalar** solo cuando el navegador confirma que la app cumple los criterios. Si el usuario ya instaló la app, el botón no aparece.

### 3.3 Ausencia de Background Sync

Safari no implementa `SyncManager`. Una cola que se registra para enviarse "cuando haya red" no se ejecuta en segundo plano en iOS. Por eso la sincronización del proyecto no depende de esa API:

- Se intenta al guardar, si `navigator.onLine` es verdadero.
- Se intenta al recibir el evento `online`.
- Se intenta al abrir la app.
- Se ofrece un botón **Sincronizar** con el número de pendientes.

El botón no es un adorno: es el camino principal en iOS y el respaldo en Android cuando el sistema decide no despertar la app.

### 3.4 Push solo con instalación

iOS 16.4 habilita Web Push para PWA agregadas a la pantalla de inicio. La app no usa notificaciones push, así que la limitación no afecta el flujo. Si se añadieran en el futuro, el requisito de instalación sería obligatorio en iOS y opcional en Android.

### 3.5 Service Worker sin módulos fiables

WebKit soporta Service Worker, pero los módulos ESM dentro del worker no son un camino confiable en todas las versiones. El proyecto compila un `sw.js` clásico con Workbox y `injectManifest`. No hay `import` en runtime dentro del worker; las dependencias se empaquetan en el build.

### 3.6 Cuota de almacenamiento opaca

Safari no expone una cuota tan predecible como Chromium. `navigator.storage.estimate()` puede devolver un valor aproximado y cambiante. La app lo usa solo para diagnóstico. El diseño no calcula "cuántos reportes caben": guarda texto y coordenadas, un volumen pequeño frente a los límites típicos. El riesgo real no es la cuota, sino la purga por ITP.

---

## 4. Particularidades de Chromium

Chrome Android ofrece el flujo más completo:

- Instalación con prompt nativo cuando se cumplen los criterios de instalabilidad.
- `navigator.storage.persist()` para solicitar almacenamiento persistente.
- Background Sync disponible, aunque el proyecto no lo use para mantener un solo flujo entre plataformas.
- Inspección de manifest, Service Worker, cachés e IndexedDB desde DevTools.

La ventaja de Chromium no cambia la arquitectura. La app usa el prompt nativo cuando existe, pero la captura, la cola y el listado son idénticos en ambos motores.

---

## 5. Solución implementada

| Capacidad | Implementación | Archivo |
|---|---|---|
| Detección de iOS | `esIOS()` revisa el agente y la plataforma | `frontend/src/main.js` |
| Detección de instalada | `esStandalone()` combina `display-mode` y `navigator.standalone` | `frontend/src/main.js` |
| Prompt Android | `beforeinstallprompt`, `deferredPrompt` y botón **Instalar** | `frontend/src/main.js` |
| Guía iOS | Diálogo con los pasos de Compartir → Agregar a inicio | `frontend/index.html`, `frontend/src/main.js` |
| Manifest | Nombre, colores, `start_url`, `scope`, íconos y atajos | `frontend/public/manifest.webmanifest` |
| Metas iOS | `apple-touch-icon`, `apple-mobile-web-app-capable` y título corto | `frontend/index.html` |
| Service Worker | Precaching, SWR, `NetworkOnly` y `offline.html` | `frontend/src/sw.js` |
| Persistencia local | Dexie sobre IndexedDB con índice por `syncStatus` | `frontend/src/db.js` |
| Solicitud de persistencia | `navigator.storage.persist()` al arrancar | `frontend/src/main.js` |
| Sincronización | Evento `online`, arranque y botón manual | `frontend/src/main.js`, `frontend/src/sync.js` |

## 6. Checklist de instalación

Criterios mínimos para que la app sea instalable. La comprobación local se hizo con Chromium y Lighthouse 11; la prueba en teléfono real se completó en Android por HTTPS con Ngrok. iOS queda implementado y documentado, pero no verificado físicamente por falta de dispositivo.

- [x] Servida por HTTPS o `localhost`. En local se usa `http://localhost:5173`, que el navegador trata como contexto seguro. En producción, TLS de Vercel y Render. Ngrok verificado en Android.
- [x] `manifest.webmanifest` enlazado con `<link rel="manifest">`. Verificado en `frontend/index.html`.
- [x] `Content-Type: application/manifest+json`. Verificado con `curl -I` sobre el servidor de producción local.
- [x] `name` y `short_name` presentes. "Reportes Técnicos de Campo" y "Reportes".
- [x] `start_url` y `scope` coherentes. Ambos apuntan a `/`.
- [x] `display: standalone` con `display_override`.
- [x] `theme_color` y `background_color`. Ambos `#f2f4f0`.
- [x] Íconos de 192×192 y 512×512 en PNG, con propósitos `any` y `maskable`.
- [x] Service Worker registrado con un handler `fetch` que responde offline.
- [x] Navegación no cacheada devuelve `offline.html`.
- [x] Lighthouse 11, categoría PWA: 100 en local y en la URL pública de Vercel.

- [x] Metas de iOS: `apple-touch-icon`, `apple-mobile-web-app-capable` y `apple-mobile-web-app-title`.
- [x] `safe-area-inset` en la navegación inferior para iPhone con indicador de inicio.
- [x] Instalación verificada en Chrome Android real por Ngrok.
- [ ] Instalación verificada en Safari iOS real: no hay dispositivo disponible. La compatibilidad está implementada y documentada, no probada físicamente.

---

## 7. Matriz de pruebas

| Navegador | Versión | Qué se prueba | Resultado esperado | Estado |
|---|---|---|---|---|
| Chrome escritorio | Reciente | Manifest, Service Worker, offline, cola, Lighthouse | Instala, guarda sin red, sincroniza, 100 en PWA | Verificado en local y producción |
| Chrome Android | Reciente | Prompt de instalación, offline, sincronización manual | Instala, guarda sin red y sincroniza al reconectar | Verificado en Android real por Ngrok |
| Safari iOS | 16.4+ | Agregar a inicio, offline, sincronización manual | Funciona tras instalar; sin prompt automático | Implementado; no verificado por falta de iPhone |
| Safari macOS | Reciente | Shell, IndexedDB y diseño | Misma captura y listado; sin prompt | No prioritario |
| Edge escritorio | Reciente | Manifest y Service Worker | Sin diferencias frente a Chrome | No prioritario |

La prueba en Android real no cambió el código: confirmó el comportamiento en hardware, la URL HTTPS y la persistencia; dos reportes capturados desde el teléfono en producción aparecen en Neon. iOS no se puede marcar como verificado: el flujo alternativo está implementado, pero una prueba física requiere un iPhone.

### Avisos que no son errores

**“Banner not shown: beforeinstallpromptevent.preventDefault() called...”**

Es un aviso informativo de Chromium. La app intercepta el evento para mostrar su propio botón **Instalar** y llama a `prompt()` cuando el usuario pulsa el botón. No es un fallo de instalabilidad y no se “arregla” permitiendo que el navegador muestre dos banners.

**“Content Security Policy of your site blocks the use of eval...”**

Vercel no envía una cabecera `Content-Security-Policy` para este sitio y el bundle de producción no contiene `eval()` ni `new Function()`. El aviso suele venir de una extensión del navegador o de una herramienta de auditoría. Para comprobarlo: abrir la app en una ventana de incógnito sin extensiones, revisar `curl -I https://frontend-chi-coral-19.vercel.app/` y buscar `eval(` en `frontend/dist`. No se debe añadir `unsafe-eval`.

---

## 8. Guion de instalación y verificación

### Android (Chrome)

1. Abrir la URL HTTPS en Chrome.
2. Esperar a que aparezca el botón **Instalar** o el aviso del navegador.
3. Instalar y abrir la app desde el icono de la pantalla de inicio.
4. Activar el modo avión, capturar un reporte y comprobar el estado **Pendiente de sincronizar**.
5. Cerrar y volver a abrir la app sin red: el reporte debe seguir ahí.
6. Desactivar el modo avión y esperar la sincronización, o pulsar **Sincronizar**.
7. Verificar el estado **Sincronizado** y la fila en `GET /api/reportes`.

### iOS (Safari)

1. Abrir la URL HTTPS en Safari.
2. Tocar **Compartir** y elegir **Agregar a inicio**.
3. Abrir la app desde el icono; debe mostrarse sin la barra de Safari.
4. Activar el modo avión, capturar un reporte y comprobar el estado pendiente.
5. Volver a abrir la app y verificar que el reporte sigue en la lista.
6. Desactivar el modo avión y pulsar **Sincronizar**.
7. Verificar el estado **Sincronizado**.

La demostración en vivo debe mostrar la URL con candado. Las capturas de contenido no sustituyen esa comprobación: sin HTTPS visible no se demuestra que el Service Worker esté activo.

---

## 9. Preguntas de defensa

**¿Por qué no usaron Background Sync?**
Porque Safari iOS no lo implementa. Habríamos tenido dos comportamientos: uno automático en Android y otro manual en iOS. Preferimos un solo flujo, con evento `online`, arranque y botón manual, disponible en ambos; Android está verificado y iOS no se probó físicamente.

**¿Qué pasa si el usuario no instala la app?**
Puede usarla en Safari, pero queda expuesto a la purga de ITP y a la incomodidad de la barra del navegador. La app muestra la guía de instalación y recomienda sincronizar pronto. No prometemos una persistencia que el navegador no garantiza.

**¿La PWA funciona sin conexión desde la primera visita?**
No. El Service Worker necesita una primera carga para precachear el shell. Desde la segunda visita, la app abre sin red. La captura offline es parte de ese modelo.

**¿Por qué el botón de sincronizar se deshabilita sin pendientes?**
Porque no hay nada que enviar. El contador de pendientes se actualiza después de cada captura y de cada sincronización. Cuando aparece un error 4xx, el reporte pasa a estado **Error al enviar** y no se reintenta en bucle.

**¿Qué pasa si el mismo reporte se envía dos veces?**
El UUID viaja desde el cliente y es la clave primaria. El servidor hace `ON CONFLICT (id) DO UPDATE` y responde 200 en el segundo intento. La fila no se duplica.

## 10. Conclusión

Chromium y WebKit comparten lo esencial: Service Worker, IndexedDB y GPS. Las diferencias que importan están en la instalación, la sincronización en segundo plano y la persistencia. La app no intenta ocultarlas con una capa común que finja paridad. Usa el prompt nativo donde existe, muestra una guía donde no, y ofrece sincronización manual en ambos casos.

La compatibilidad se sostiene en tres decisiones: un `sw.js` clásico, una cola que no depende de Background Sync y una instalación guiada en iOS. Android quedó verificado en hardware; iOS queda implementado y documentado, pero no verificado físicamente por falta de dispositivo.